

ALGORITHM RACE DASHBOARD

Interactive Performance Comparison of Sorting and Searching Algorithms

A Design and Analysis of Algorithms (DAA) Project Report

Algorithms: Merge Sort, Quick Sort, Heap Sort, Binary Search

Technologies: Python, Streamlit, Plotly

1. Problem Statement

The Algorithm Race Dashboard is designed to allow users to compare the performance of different algorithms on identical datasets.

The system measures execution time and algorithmic operations, demonstrates Binary Search on sorted data, and explains why observed practical performance may differ from theoretical complexity.

2. Objectives

Implement fundamental Design and Analysis of Algorithms techniques.

Compare Merge Sort, Quick Sort, and Heap Sort using the same datasets.

Implement Binary Search on sorted data.

Measure execution time and comparisons/search steps.

Visualize algorithm performance using interactive graphs.

Help users understand the relationship between theoretical and practical algorithm performance.

3. Core Algorithms

Merge Sort: A divide-and-conquer sorting algorithm with $O(n \log n)$ best, average, and worst-case time complexity.

Quick Sort: A divide-and-conquer sorting algorithm with $O(n \log n)$ average complexity and $O(n^2)$ worst-case complexity, depending on pivot selection and input.

Heap Sort: A heap-based sorting algorithm with $O(n \log n)$ best, average, and worst-case time complexity.

Binary Search: A searching algorithm for sorted data that repeatedly halves the search interval, with $O(\log n)$ average and worst-case complexity and $O(1)$ best-case complexity.

4. System Architecture

User → Streamlit Interface → Dataset Generator → Sorting Algorithms → Performance Measurement → Plotly Visualization.

Binary Search follows: Dataset → Sorting → Sorted Data → Binary Search → Search Steps/Comparisons.

5. Methodology

1. Generate or provide a dataset.
2. Select the required dataset size and type.
3. Create identical copies of the dataset for fair comparison.
4. Run Merge Sort, Quick Sort, and Heap Sort.
5. Measure execution time and relevant comparisons/operations.
6. Sort the data before performing Binary Search.
7. Enter a target value and perform Binary Search.
8. Record the number of search steps/comparisons.
9. Display results in tables and interactive graphs.

6. Technologies Used

Python: Used for algorithm implementation, dataset generation, performance measurement, and data processing.

Streamlit: Used to build the interactive web-based dashboard and collect user inputs.

Plotly: Used to create interactive charts for comparing execution time, input size, and other performance metrics.

7. Performance Analysis

Theoretical complexity describes how an algorithm's resource requirements grow as input size increases, but it does not specify an exact execution time on a particular computer.

Practical performance can differ because of hardware, Python interpreter overhead, implementation details, memory access patterns, recursion overhead, input distribution, Quick Sort pivot selection, and background system activity.

Therefore, the dashboard compares measured results with theoretical complexity to help users understand both perspectives.

8. Expected Results and Advantages

Expected results include increasing execution time with increasing input size, predictable $O(n \log n)$ behavior for Merge Sort and Heap Sort, input-dependent behavior for Quick Sort, and logarithmic search behavior for Binary Search.

Advantages include an interactive interface, fair comparison using identical datasets, clear visualizations, educational value, and a direct connection between theoretical analysis and practical experimentation.

9. Future Scope

Add more algorithms such as Bubble Sort, Insertion Sort, Selection Sort, Radix Sort, and Counting Sort.

Add step-by-step sorting animations and visualizations.

Visualize Quick Sort pivot selection and Heap Sort as a tree.

Add memory-usage analysis and multiple-run statistical analysis.

Allow users to export performance results and reports.

10. Conclusion

The Algorithm Race Dashboard provides an interactive platform for understanding and comparing important sorting and searching algorithms.

The project demonstrates that algorithm performance depends not only on theoretical complexity but also on implementation, hardware, input characteristics, and runtime conditions.

Python, Streamlit, and Plotly provide a practical technology stack for building an understandable and educational DAA performance-analysis tool.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, MIT Press.
- Python Documentation: <https://docs.python.org/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- Plotly Python Documentation: <https://plotly.com/python/>